# Software Development for Robotics (SDfR) - Assignment IV

S.T.C. Fokkema (s2710714) & J.B.E. Nwandu (s3537544) – PPD: Group 1

March 11, 2026

## File Structure

```
PPD-01-SDfR-Assignment4.zip
└── assignment4/
    └── assignment4.1/
        ├── include/
        │   ├── internationalPackage.hpp
        │   ├── overnightInternationalPackage.hpp
        │   ├── overnightPackage.hpp
        │   ├── package.hpp
        │   ├── retourPackage.hpp
        │   ├── sameDayPackage.hpp
        │   └── twoDayPackage.hpp
        ├── src/
        │   ├── internationalPackage.cpp
        │   ├── main.cpp
        │   ├── overnightInternationalPackage.cpp
        │   ├── overnightPackage.cpp
        │   ├── package.cpp
        │   ├── retourPackage.cpp
        │   ├── sameDayPackage.cpp
        │   ├── test.cpp
        │   └── twoDayPackage.cpp
        ├── ClassHierarchyDiagram.png
        ├── ClasshierarchyDiagram.uxf
        └── README.txt
```

Figure 1: Complete file structure of Assignment 4.

## Class Hierarchy Diagram

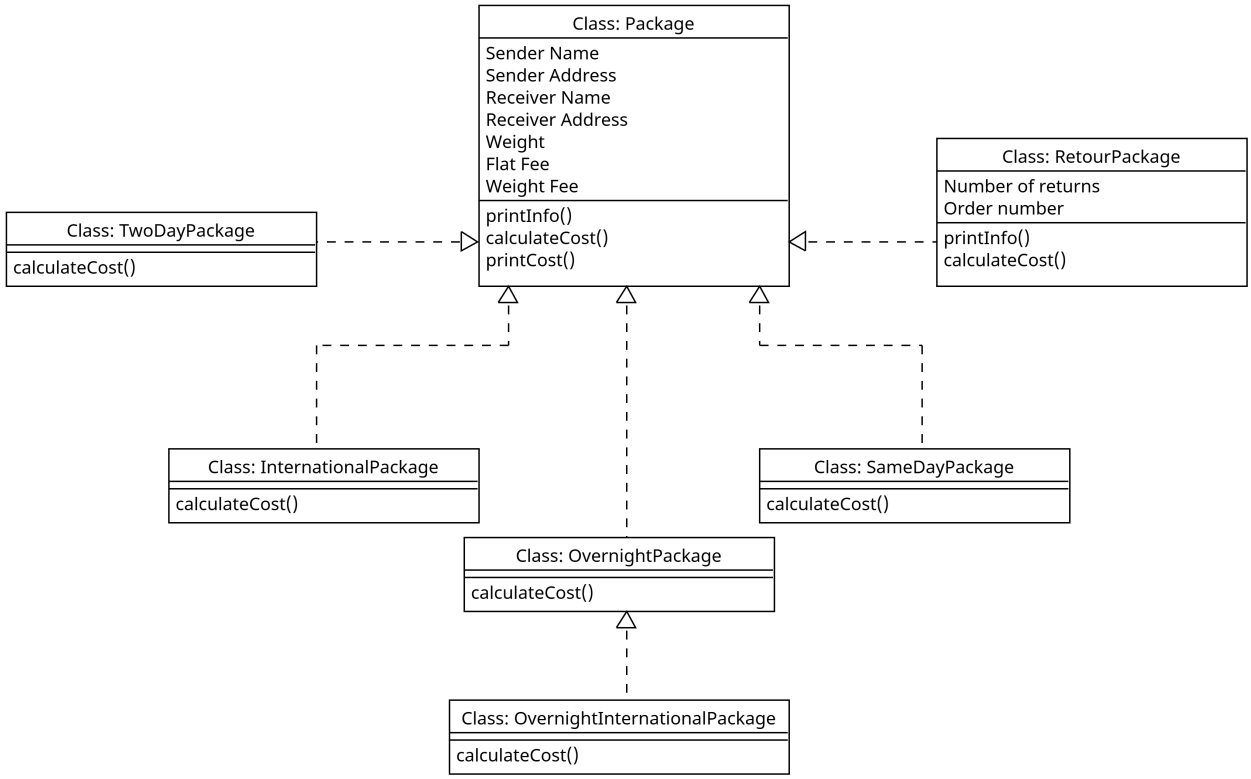



Figure 2: Class Hierarchy Diagram, made using UMLet in VS Code.

# 1 Results

A total of 6 different package classes were created, these are:

- twoDayPackage
- overnightPackage
- internationalPackage
- sameDayPackage
- overnightInternationalPackage
- retourPackage

and, of course, the base "package" class at the top of the inheritance structure. 4 of the above mentioned classes directly inherit everything from the base class, only overwriting the cost function to add additional calculations on top of the base cost. The `overnightInternationalPackage` instead inherits directly from the `overnightPackage`, and also only updates the cost function to add an additional 21% charge on top of the overnight package cost. Finally, the `retourPackage` adds two new parameters: `numberOfReturns` and `orderNumber`. It then updates the `printInfo` function to also include these two parameters on top of the regular print info, and loosely bases its cost function on the amount of returns. Figure 3 shows the output

of every package type, with the same parameters as input: weight = 10, flatFee = 5 and weightFee = 2.5, with numberOfReturns = 2 for the `retourPackage`.

```
Two Day Package Info:
Sender: Richi, Mandarijn Straat
Receiver: Vlad, Lekker Gezellig Straat
Weight: 10 kg
Cost: 30 euros

Same Day Package Info:
Sender: Jordan, Calslaan
Receiver: Steyn, Matenweg
Weight: 10 kg
Cost: 180 euros

Overnight Package Info:
Sender: Tomoki, 123 Main St
Receiver: Sjoerd, 456 Elm St
Weight: 10 kg
Cost: 140 euros
```

```
International Package Info:
Sender: Ira, Teletubbieland
Receiver: Malta, 742 Evergreen Terrace
Weight: 10 kg
Cost: 36.3 euros

Overnight International Package Info:
Sender: Alice, Wonderland
Receiver: Bob, Builder Street
Weight: 10 kg
Cost: 169.4 euros

Retour Package Info:
Retour package with order number 2026001
Number of items in return: 2
Sender: Charlie, 123 Oak St
Receiver: David, 456 Pine St
Weight: 10 kg
Cost: 40 euros
```

Figure 3: Output of the code for assignment 4, with the same input arguments.

## 2 Design Choices

The first design choice of note is placing the given information (such as `senderName`) under *protected* in the `package` class. This way, these parameters are only accessible in this class and its children, and this prevents them from being edited outside of these classes. They become "encapsulated". While this is not strictly necessary for this assignment, it is good coding practice.

As for the functions in the class, both `printInfo` and `calculateCost` get the 'virtual' prefix, while the `printCost` function does not. This is because the `printCost` function stays identical for every child class, while the other two, especially `calculateCost`, can have slight differences that are still based on the original function.

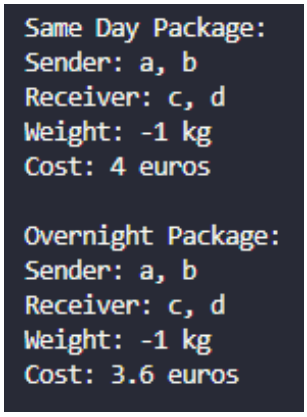
Next, the `overnightInternationalPackage` and `retourPackage` classes. These two classes were added specifically to try out different inheritance features. Whilst the former inherits from an already inherited class, the latter adds additional parameters of its own.

## 3 Testing

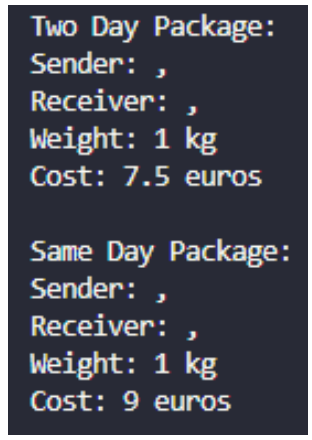
For testing, a new `test.cpp` file was created that takes user input for the strings so that different cases can be tested quickly. Four different edge cases have been tested: setting every numerical value to 0, setting every numerical value to a negative number (-1), leaving every string empty, and leaving the weight empty. Figure 4 shows the results of these four tests. The first three behave just as expected: 0 weight returns only the flat fee, and in the case of the international package it returns the flat fee * 1.21. The negative weight subtracts the weight fee from the flat fee, instead of adding to it. And the empty strings simply don't display any text. However, leaving the weight empty causes something interesting: the program simply stop

running, it freezes in place. This happens because the `std::cin` command is still waiting for an input, the program will simply not continue until it receives something. So, in this test case, it is impossible to actually keep running the program without entering a value; it is required.

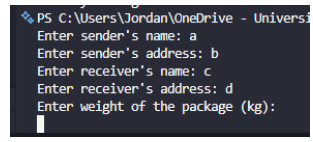
(a) Every numerical value to 0.




(b) Every numerical value to -1.




(c) Every string left empty.




(d) Weight left empty.

Figure 4: Output from test cases.

# 4 Questions

(a) The inheritance is used because both sub-packages have all of the functionalities of the regular parent Package. Using inheritance avoids having duplicate code, which may later not be updated with the parent code, which may lead to errors.

(b) Protected variables are accessible to children classes that inherit from the parent. In our example, the children classes need access to the cost of the parent class (in order to give this a value through `calculateCost`), so the cost is made protected, rather than private. Private variables are not accessible by the children classes.

(c) To describe composition and inheritance, we describe their difference. Inheritance can be seen as an "is-a" relation, e.g., a laptop is a computer. When describing computers, you would say that a laptop inherits the properties of a computer. Composition can be seen as a "has-a" relationship, e.g., a computer (and therefore a laptop) has a CPU, RAM, Monitor, and many other components (which can be described by classes) that together make up the computer.

(d) Destructors are called at the end of the scope of every segment of code. For each segment of code such as a function, all variables that are defined within {} are destroyed when the } is reached to free up memory. Every global variable that is defined outside of any scope has their destructor called during program termination.

(e) The keyword *virtual* can be added to destructors to ensure that when there is any ambiguity on whether a certain object is of the child or parent type, when the destructor for this object is called, the correct destructor will be used. For example, you can create a pointer to a parent class, which refers to a child class. This child class inherits from the parent, and therefore it is the parent class. However, when this pointer's destructor is called in the usual way, the parents destructor will be used, which might not deallocate the memory of certain variables specific to the child.

## AI Disclosure

During the preparation of this work the authors used `MS Copilot` in order to improve code and syntax. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the delivered work.